ECE 2218 Lab #7 - Motor Control of the Pololu 3pi Robot using Pulse-Width Modulation

Introduction

In this lab you will write programs for the Pololu 3pi robot with GCC and Microchip Studio. The goal is to control the robot's two motors in both a simple, full speed, on / off mode and in a variable speed / direction mode using pulse-width modulation. You need to study the documents describing the Pololu Robot prior to the class. The schematic will probably be most useful. The tables on page 59 and 60 of the Pololu 3pi+ 32U4 User's Guide will also be a good reference.



Before the Lab

Write and test the motor control program. The program should do the following:

1. Setup
    a. Write a routine to read from Switch A (PB3) and loop indefinitely until the switch is pressed. You don't want the program to start as soon as it is programmed. You may need time to (gently) unplug the USB programming cable.
    b. Set up proper pins on Port B for output on the motor control pins. Identify which functions for the motor controller go to the different Port B pins.
2. Full speed test one motor at a time.
    a. Turn on the left motor full-speed forward for 1s while the right motor is braked (off).
    b. Brake both the motors and wait a few seconds.

    c. Turn on the left motor full-speed reverse for 1s while the right motor is braked (off).
    d. Brake both the motors and wait a few seconds.
    e. Turn on the right motor full-speed forward for 1s while the left motor is off.
    f. Brake both the motors and wait a few seconds.
    g. Turn on the right motor full-speed reverse for 1s while the left motor is off.
    h. Brake both the motors and wait a few seconds.
3. Full speed test both motors together.
    a. Make the robot go full-speed forward for 1s.
    b. Brake both the motors and wait a few seconds.
    c. Make the robot go full-speed reverse for 1s.
    d. Brake both the motors and wait a few seconds.
    e. Make the robot spin in place at full speed CCW (counterclockwise) for 1s.
    f. Brake both the motors and wait a few seconds.
    g. Make the robot spin in place at full speed CW (clockwise) for 1s.
    h. Brake both the motors and wait a few seconds.
4. PWM control is accomplished using the Output Compare pins for Time 1 (a 16-bit timer). Configure the timer for Phase Correct PWM generation on OC1A (right motor) and OC1B (left motor). We will need to experiment with the prescaler values. If the PWM frequency is too fast the motor may not respond correctly. Too slow and it may be jerky.
    a. Make the robot go ¼-speed forward for 2s.
    b. Brake both the motors and wait a few seconds.
    c. Make the robot go ¼-speed reverse for 2s.
    d. Brake both the motors and wait a few seconds.
    e. Make the robot spin in place CCW at ¼-speed (counterclockwise) for 2s.
    f. Brake both the motors and wait a few seconds.
    g. Make the robot spin in place CW at ¼-speed (clockwise) for 2s.
    h. Brake both the motors and wait a few seconds.

Unfortunately, the simulator for this microcontroller is not implemented in Microchip Studio. However, the differences between the 32U4 and the 328P are small, so if desired you may test out your algorithms by writing them for the 328P and using the simulator.

In the Lab

1. Test out the OLED display

   Copy the provided font.c, OLED.c and OLED.h files to your program. Add them to your project by going to the Solution Explorer window, right click on the Project and Add -> Existing Item.

In your program, add #include “OLED.h” near the top. This will tell the compiler where to find the definitions of the functions we will be using. Also add `#include <stdio.h>` for the snprintf() function.

Next, inside your main() function add:

```
char text[50];  // define an array to hold text to be written to the display
OLED_Init();  // set up the pins for input/output that connect to the display
OLED_reset(); // toggle the display’s reset pin
OLED_clearDisplayRam();  // display data is held in RAM.  Zero everything to
blank display
OLED_configureDefault();  // set mode, contrast, etc
snprintf(text,8,"%s","ECE2218");  // copy a string constant into our variable
OLED_writeText(text);  // display the string contents
```

Compile and run the program. Read the documentation for the snprintf function. **Note that this is a very simplistic use of the OLED-writeText() function and may require modification.**

2. Program your robot and test it out. Use the OLED_writeText() function to display values for debugging purposes.

3. Find the best prescaler value.

   Experiment with the clock prescaler value for the timer. Run the previous program 5 times, each with one of the 5 different prescaler possibilities. Make notes about the behavior of the robot with each setting. Decide which one is the best to use. Explain your decision.

After the Lab

Make listings of your corrected, annotated programs. Comment on any unexpected behaviors exhibited by the robot during the experiments. Explain the difference in behavior between non-inverting and inverting control methods. Describe the results of part 3 from the in-lab portion for each of the prescaler values. Justify your choice of the best value to use.